hashlock.

Security Audit

Neomi (Auction)

September 2024

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	16
Conclusion	17
Our Methodology	18
Disclaimers	20
About Hashlock	21

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Neomi team partnered with Hashlock to conduct a security audit of their NeomiDonation.sol smart contract. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

Neomi is an investment platform focused on enabling impact-driven projects by simplifying capital raising and promoting decentralized finance (DeFi). It helps businesses create, manage, and exchange digital assets while aligning investments with ethical values. Neomi provides services for both issuers and investors, offering tools for creating tokenized assets, managing investor relations, and automating compliance and governance processes. Their goal is to make impact investments more transparent, accessible, and effective.

Neomi also promotes sustainable investing through its "Halo Club," which supports impact entrepreneurs and projects in gaining financial backing. The platform is designed for those looking to make positive contributions to society through their investments

Project Name: Neomi
Compiler Version: ^0.8.26
Website: www.neomi.io
Logo:

Visualised Context:

Project Name
Neomi

Launch Date
TBA

Compiler Version
v^0.8.26

Language
Solidity

Network
EVM

Token Ticker
TBA

Project Visuals:

Audit scope

We at Hashlock audited the solidity code within the Neomi project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Neomi Smart Contract**
Platform	**EVM / Solidity**
Audit Date	**September, 2024**
Contract	NeomiDonation.sol
Contract MD5 Hash	55f6e5430a0a6ed189e6ebbbafcd2636

Security Rating

After Hashlock's audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

1 Medium-severity vulnerabilities

1 Low-severity vulnerabilities

2 QAs

2 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour

NeomiDonation.sol
- Allows a beneficiary to:
    - Schedule an auction
    - Set a token URI
    - Mint NFTs
    - Withdraw the funds from the contract
    - Extend an auction
- Allows users to:
    - Bid auctions
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the Neomi project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Neomi project smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

Medium

[M-01] NeomiDonation#scheduleAuction - Allow to schedule a new auction before the auction of the currentIndex begins

Description

The `scheduleAuction` function allows the beneficiary to schedule a new auction before the auction of the `currentIndex` begins.

Vulnerability Details

The `scheduleAuction` function only checks if the auction of the `currentIndex` is active and does not check if the auction began.

```
function scheduleAuction(uint256 startTime, uint256 endTime, string memory tokenUri)
public onlyBeneficiary {
        . . .
        require(
            !isAuctionActive(currentIndex) && !isAuctionScheduled(currentIndex + 1),
            "An auction is already in progress or scheduled."
        );
        . . .
}
```

Impact

It can result in scheduling a new auction before the auction of the `currentIndex` begins, and further making 2 active auctions, which could further disrupt the system and lead to unpredictable behavior in the contract's auction logic.

Recommendation

It's recommended that the `scheduleAuction` only allows to schedule a new auction after the auction of the `currentIndex` has ended.

Status

Resolved

Low

[L-01] NeomiDonation#setTokenURI - Lack of emitting an event

Description

The `setTokenURI` function is missing an event when key storages are updated.

Recommendation

Add a relevant event in the function.

Status

Resolved

QA

[Q-01] NeomiDonation#scheduleAuction - Unnecessary checks

Description

The `scheduleAuction` function has 2 unnecessary checks.

- At the beginning of the function, it checks `endTime - startTime >= MIN_AUCTION_DURATION` and it means `endTime` should be greater than `startTime`. So no need to check `endTime > startTime` later.

- `isAuctionScheduled(currentIndex + 1)` is always false because `currentIndex` is already updated to the value of the auction index to be scheduled.
  So there is no scheduled auction at `currentIndex + 1`.

Recommendation

Remove such unnecessary checks in the function.

Status

Resolved

[Q-02] NeomiDonation - Unused import

Description

The `NeomiDonation` contract inherits the `ReentrancyGuard` library but the contract doesn't use any modifiers from the `ReentrancyGuard` library.

Recommendation

Remove the unused import.

Status

Resolved

Gas

[G-01] NeomiDonation#mintNFT - Storage variable is read multiple times

Description

The `mintNFT` function reads `auctions[auctionIndex].highestBidder` from storage multiple times.

Recommendation

Cache the `auctions[auctionIndex].highestBidder` into a memory variable.

Status

Acknowledged

[G-02] NeomiDonation - `beneficiary` could be made immutable

Description

The `beneficiary` variable is only set in the constructor.

Recommendation

Make the `beneficiary` variable immutable.

Status

Acknowledged

Centralisation

The Neomi project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the Neomi project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:
In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:
Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au